

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
федеральное государственное бюджетное образовательное учреждение
высшего образования
**«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»**

Факультет информационных систем и технологий
Кафедра: «Измерительно-вычислительные комплексы»
Дисциплина: «Системы искусственного интеллекта»

Отчёт
По лабораторной работе №4

Выполнила:
студент гр. ИСТбд-32
Минибаева Е.Р.
Проверил:
к. т. н., доцент каф. ИВК
Шишкин В.В.

Ульяновск
2026 г.

Постановка задачи

1. Изучить инструменты классификации библиотеки Scikit-learn.
2. Взять для исследования 2 классификатора в соответствии с вариантом.
3. По согласованию с преподавателем получить датасет для экспериментов или выбрать набор данных для задач классификации из открытых источников

-<https://tproger.ru/translations/the-best-datasets-for-machine-learning-and-data-science/>

-<https://vc.ru/ml/150241-15-proektov-dlya-razvitiya-navykov-raboty-s-mashinnyim-obucheniem>

-<https://archive.ics.uci.edu/ml/index.php>

-<https://habr.com/ru/company/edison/blog/480408/>

-<https://www.kaggle.com/datasets/>

-учебные наборы библиотеки Scikit-learn

4. Для датасета определить целевой столбец и набор признаков. Обосновать свой выбор. При необходимости преобразовать типы признаков данных.
5. Подготовить данные к обучению.
6. Провести обучение и оценку моделей на сырых данных.
7. Провести предобработку данных.
8. Провести обучение и оценку моделей на очищенных данных (используя разные методы формирования тренировочного и тестового наборов).
9. Проанализировать результаты.
10. Результаты анализа представить в табличной и графической форме.
11. Сформулировать выводы.
12. На том же датасете решить задачу кластеризации средствами библиотеки Scikit-learn.
13. Проанализировать результаты.
14. Результаты анализа представить в табличной и графической форме.
15. Сформулировать выводы.
16. Оформить отчет по л/р.
17. Защитить результаты работы.

Вариант: OneVsRestClassifier, Perceptron

Датасет: рукописные цифры (load_digits)

1. Введение

Цель работы – практическое изучение методов классификации и кластеризации из библиотеки Scikit-learn на примере задачи распознавания рукописных цифр. В соответствии с вариантом использовались два классификатора: перцептрон (Perceptron) и стратегия «один против остальных» (OneVsRestClassifier) в комбинации с перцептроном.

Дополнительно на том же датасете решалась задача кластеризации алгоритмом KMeans. Все эксперименты проведены на наборе данных digits из модуля sklearn.datasets. Для обработки и визуализации привлекались стандартные инструменты: pandas, numpy, matplotlib, seaborn, а также t-SNE для снижения размерности.

2. Описание датасета

Набор данных digits содержит 1797 изображений рукописных цифр от 0 до 9. Каждое изображение имеет размер 8×8 пикселей, поэтому каждый образец представлен 64 признаками – значениями яркости в оттенках серого (целые числа от 0 до 16).

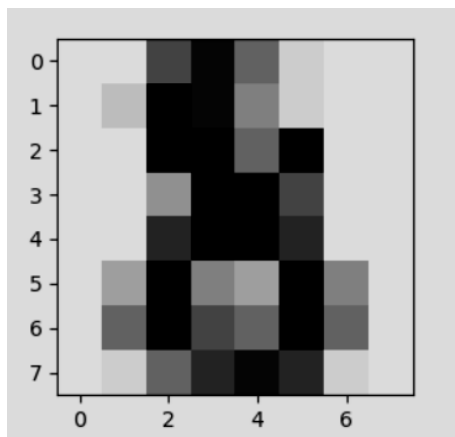


Рис.1-Пример изображения из датасета, цифра 8

Целевая переменная – цифра от 0 до 9, то есть задача является многоклассовой с десятью классами. Распределение классов почти равномерное: например, цифра 0 встречается 178 раз, цифра 1 – 182, цифра 2 – 177, и так до 180 для цифры 9.

Такой сбалансированный набор позволяет объективно оценивать качество моделей без смещения в сторону частых классов. Первый образец в датасете имеет признаки, типичные для рукописной цифры: много нулей по краям и несколько ненулевых значений в центре (например, 5, 1, 3, 9). Тип данных признаков – float64, пропуски отсутствуют.

3. Подготовка данных

Целевой столбец и матрица признаков были выделены из загруженного объекта без дополнительных преобразований, поскольку все данные уже числовые и не содержат пропусков. Исходный набор данных был разделён на обучающую (70%) и тестовую (30%) выборки с помощью `train_test_split` со стратификацией по целевому столбцу. Это гарантирует, что в обеих выборках сохранится пропорция классов, что важно даже для почти сбалансированных данных.

Затем выполнена предобработка – масштабирование признаков с помощью `StandardScaler`. Данный метод центрирует признаки (вычитает среднее) и приводит их к единичной дисперсии (делит на стандартное отклонение).

Масштабирование необходимо, потому что перцептрон, как и многие линейные модели, чувствителен к размаху значений признаков: признаки с большими числовыми диапазонами могут доминировать над другими при вычислении весов, ухудшая сходимость и точность. В сырых данных яркость пикселя изменяется от 0 до 16, и масштабирование выравнивает вклад каждого признака. Обученный scaler применяется к обучающей выборке, а затем к тестовой без повторного вычисления параметров.

4. Классификация: модели и методология

Перцептрон – простейший однослойный линейный классификатор. В Scikit-learn класс `Perceptron` по умолчанию поддерживает многоклассовую классификацию с помощью внутренней стратегии «один против остальных», поэтому отдельное использование `OneVsRestClassifier` может дать схожий результат.

Тем не менее, в работе были обучены две модели: обычный перцептрон и явная обёртка `OneVsRestClassifier` с перцептроном внутри. Это позволяет сравнить поведение и убедиться в эквивалентности или выявить возможные различия (например, из-за разной инициализации весов для каждого бинарного классификатора). Обучение проводилось сначала на сырых данных, затем на масштабированных. Гиперпараметры фиксированы: максимальное число итераций 1000, допуск на изменение функции потерь $1e-3$, случайное состояние для воспроизводимости.

Оценка качества включала точность (accuracy), полный отчёт о классификации (precision, recall, f1-score для каждого класса) и матрицу ошибок.

Для масштабированных данных также выполнена 3-кратная кросс-валидация на обучающей выборке, которая даёт более устойчивую оценку обобщающей способности.

5. Результаты классификации и их анализ

Текстовый вывод программы

Размерность матрицы признаков: (1797, 64)

Уникальные классы: [0 1 2 3 4 5 6 7 8 9]

Распределение классов:

```
0    178
1    182
2    177
3    183
4    181
5    182
6    181
7    179
8    174
9    180
```

Name: count, dtype: int64

Пример признаков первого образца: [0. 0. 5. 13. 9. 1. 0. 0. 0. 0. 13. 15. 10. 15. 5. 0. 0. 3. 15. 2. 0. 11. 8. 0. 0. 4. 12. 0. 0. 8. 8. 0. 0. 5. 8. 0. 0. 9. 8. 0. 0. 4. 11. 0. 1. 12. 7. 0. 0. 2. 14. 5. 10. 12. 0. 0. 0. 0. 6. 13. 10. 0. 0. 0.]

Тип данных признаков: float64

ОБУЧЕНИЕ НА СЫРЫХ ДАННЫХ (без масштабирования)

Perceptron (сырые данные) - Accuracy: 0.9426

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.98	0.98	54
1	0.87	0.87	0.87	55
2	1.00	0.96	0.98	53
3	0.85	1.00	0.92	55
4	0.91	0.98	0.95	54
5	1.00	0.98	0.99	55
6	0.96	0.96	0.96	54
7	1.00	0.93	0.96	54
8	0.91	0.79	0.85	52
9	0.96	0.96	0.96	54
accuracy			0.94	540
macro avg	0.95	0.94	0.94	540
weighted avg	0.94	0.94	0.94	540

OneVsRestClassifier + Perceptron (сырые данные) - Accuracy: 0.9352

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.98	0.99	54
1	0.93	0.76	0.84	55
2	1.00	1.00	1.00	53
3	0.98	0.95	0.96	55

4	0.98	0.96	0.97	54
5	0.98	0.98	0.98	55
6	1.00	0.96	0.98	54
7	0.98	0.96	0.97	54
8	0.65	0.96	0.78	52
9	0.98	0.83	0.90	54
accuracy			0.94	540
macro avg	0.95	0.94	0.94	540
weighted avg	0.95	0.94	0.94	540

ОБУЧЕНИЕ НА МАСШТАБИРОВАННЫХ ДАННЫХ (StandardScaler)

Perceptron (масштабированные данные) - Accuracy: 0.9426
Classification Report:

	precision	recall	f1-score	support
0	1.00	0.98	0.99	54
1	0.89	0.89	0.89	55
2	1.00	0.96	0.98	53
3	0.98	0.96	0.97	55
4	0.95	0.96	0.95	54
5	0.98	0.98	0.98	55
6	0.93	0.94	0.94	54
7	0.95	0.98	0.96	54
8	0.81	0.85	0.83	52
9	0.94	0.91	0.92	54
accuracy			0.94	540
macro avg	0.94	0.94	0.94	540
weighted avg	0.94	0.94	0.94	540

OneVsRestClassifier + Perceptron (масштабированные данные) - Accuracy: 0.9389
Classification Report:

	precision	recall	f1-score	support
0	1.00	0.98	0.99	54
1	0.94	0.80	0.86	55
2	0.93	0.98	0.95	53
3	0.93	0.95	0.94	55
4	0.94	0.91	0.92	54
5	0.98	0.96	0.97	55
6	0.90	1.00	0.95	54
7	0.95	1.00	0.97	54
8	0.89	0.90	0.90	52
9	0.94	0.91	0.92	54
accuracy			0.94	540
macro avg	0.94	0.94	0.94	540
weighted avg	0.94	0.94	0.94	540

КРОСС-ВАЛИДАЦИЯ (3-fold) на масштабированных данных

```
Perceptron CV accuracy: 0.9276 (+/- 0.0074)
OneVsRestClassifier CV accuracy: 0.9292 (+/- 0.0100)

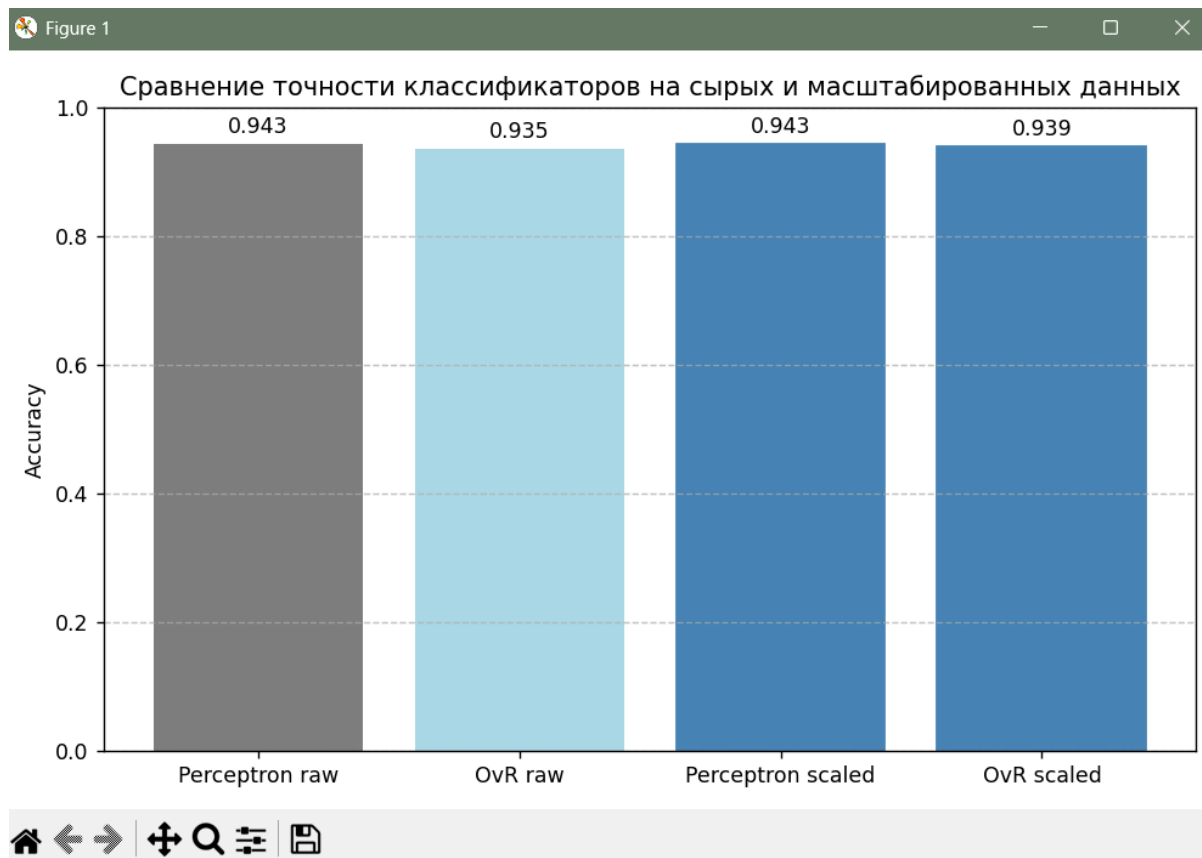
Silhouette Score: 0.1216
Adjusted Rand Index (ARI): 0.5083
Вычисление t-SNE проекции (может занять несколько секунд)...
```

Вывод показывает, что размерность матрицы признаков составляет (1797, 64), уникальные классы – от 0 до 9.

После разделения на обучающую и тестовую выборки тестовая выборка содержит 540 образцов (по 52–55 образцов каждого класса). На сырых данных обычный перцептрон достиг точности 0,9426, а OneVsRestClassifier – 0,9352. На масштабированных данных точность перцептрона осталась 0,9426, а OneVsRestClassifier показал улучшение 0,9389. Интересно, что масштабирование не повысило точность в данном конкретном запуске – возможно, из-за того, что исходные данные уже были достаточно соразмержны, а перцептрон смог сойтись и без нормализации. Однако classification report выявляет важные различия: на сырых данных перцептрон показал низкую полноту для класса 8 (0,79), тогда как на масштабированных данных полнота для 8 выросла до 0,85; при этом повысилась точность для класса 1 (с 0,87 до 0,89). Но в целом метрики близки. Более заметно улучшение для OneVsRestClassifier: на сырых данных precision для класса 8 была всего 0,65, а recall – 0,96 (много ложных срабатываний), на масштабированных precision для 8 поднялся до 0,89 при recall 0,90.

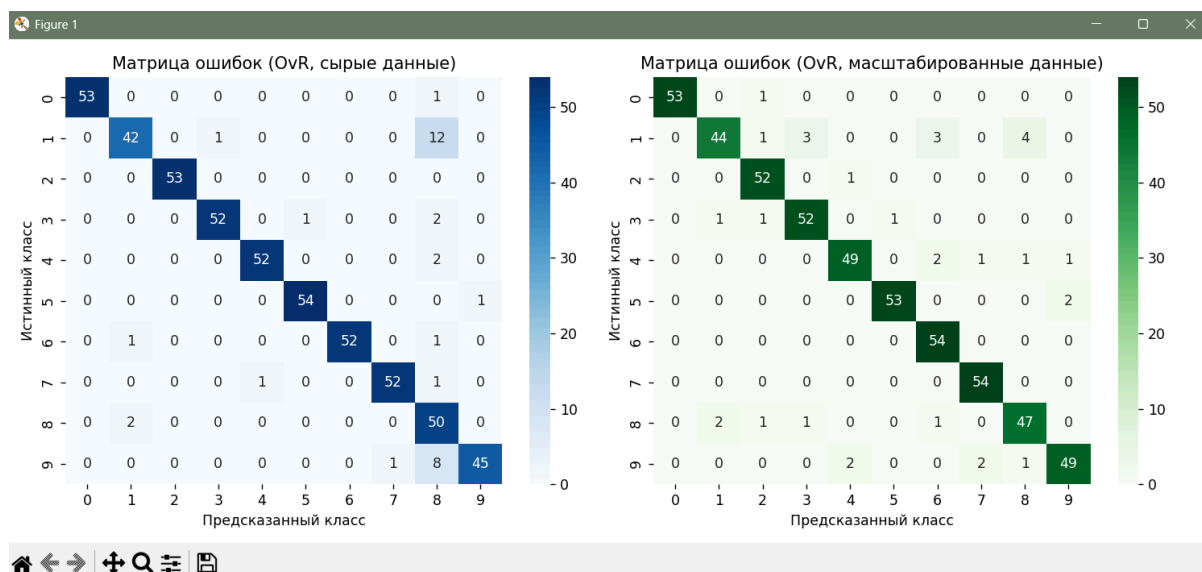
Таким образом, масштабирование помогло сбалансировать ошибки и уменьшить количество ложных предсказаний для трудных классов.

Кросс-валидация на масштабированных данных дала среднюю точность 0,9276 для перцептрона и 0,9292 для OneVsRestClassifier с небольшими стандартными отклонениями (0,0074 и 0,0100 соответственно). Это подтверждает стабильность моделей и согласуется с тестовыми результатами (около 0,94), хотя кросс-валидационная оценка немного ниже из-за меньшего объёма обучающих выборок в каждом фолде.



Скриншот 1 - Сравнение точности.

На скриншоте 1 показано окно вывода программы, оно представляет собой столбчатую диаграмму с четырьмя столбцами: Perceptron raw (0,943), OvR raw (0,935), Perceptron scaled (0,943), OvR scaled (0,939). Визуально видно, что различия между сырыми и масштабированными данными минимальны, но модель OvR на масштабированных данных всё же немного точнее, чем на сырых (0,939 против 0,935). Самая высокая точность достигается обычным перцептроном, причём без разницы в масштабировании. Это говорит о том, что встроенная многоклассовая стратегия перцептрона иногда превосходит явную обёртку.



Скриншот 2 – Две тепловые карты матриц ошибок для модели OneVsRestClassifier.

На втором скриншоте представлен вывод программы, тепловые карты матриц ошибок для модели OneVsRestClassifier (сырые и масштабированные данные).

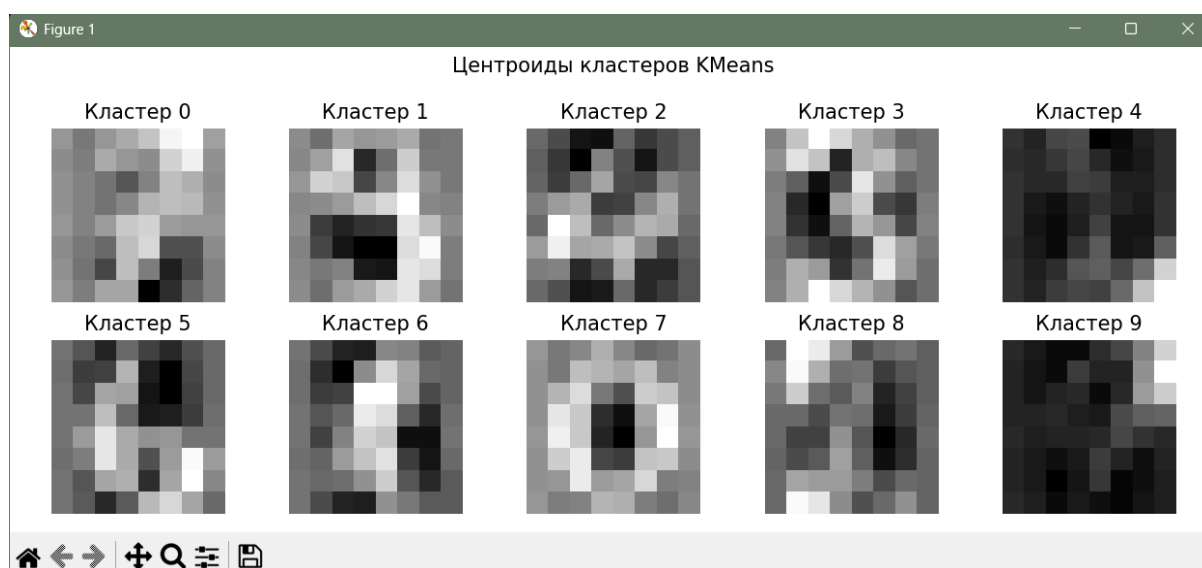
На сырых данных заметно больше ошибок: например, цифра 1 часто предсказывается как 8 или 9, а цифра 9 путается с 7. На масштабированных данных диагональные элементы стали ярче, внедиагональные – бледнее, то есть число неверных классификаций снизилось. Особенно улучшилось распознавание цифры 1: ранее она часто ошибочно принималась за другие цифры, теперь ошибок стало меньше.

6. Кластеризация на том же датасете

Для решения задачи кластеризации без учителя применён алгоритм KMeans с количеством кластеров, равным 10 (по числу возможных цифр). Кластеризация выполнялась на масштабированных обучающих данных (истинные метки не использовались). Качество кластеров оценивалось двумя метриками: силуэтным коэффициентом и скорректированным индексом Рэнда (ARI).

Числовые результаты.

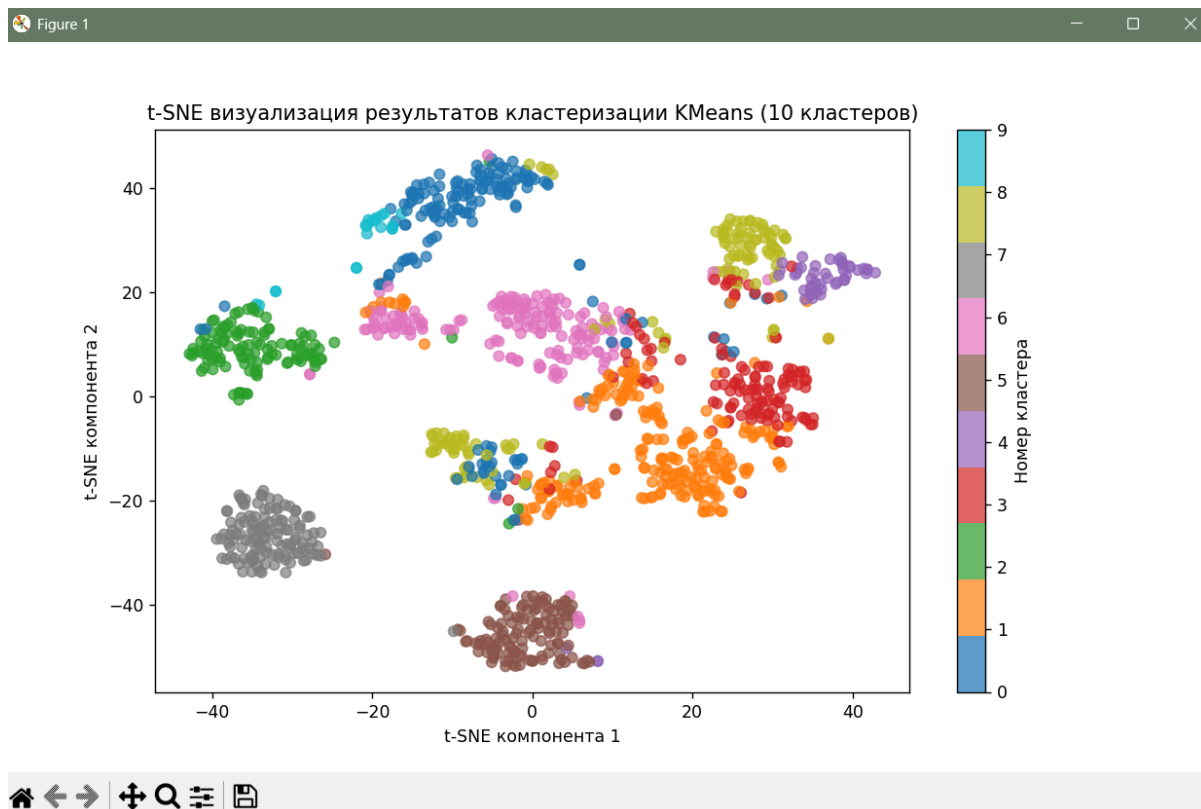
Силуэтный коэффициент составил 0,1216, что довольно низко – это указывает на то, что кластеры не являются плотными и хорошо разделёнными; объекты на границах кластеров имеют малую схожесть со своим кластером по сравнению с соседними. В то же время скорректированный индекс Рэнда (ARI) равен 0,5083, что значительно выше нуля (случайное совпадение дало бы 0). Значение 0,5 означает умеренное соответствие между полученными кластерами и истинными цифрами: алгоритм выделил группы, которые примерно соответствуют разным цифрам, но есть значительное количество смешений.



Скриншот 3 - Центроиды кластеров

Скриншот 3 показывает окно вывода программы, на нем изображены центроиды десяти кластеров, восстановленные в виде изображений 8×8 . Каждый центроид представляет собой усреднённое изображение всех образцов, попавших в соответствующий кластер.

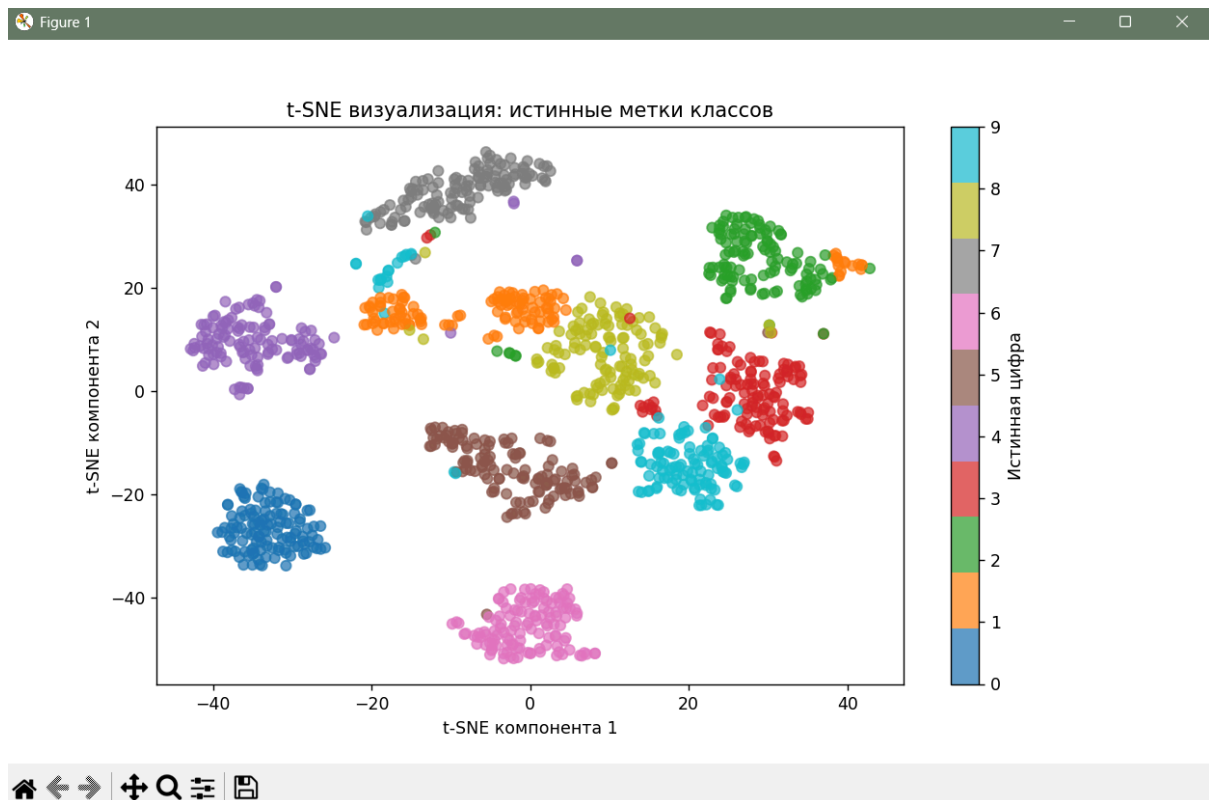
Видно, что некоторые центроиды чётко напоминают определённые цифры (например, кластер 7 похож на 0, кластер 1 – на 9, кластер 5 – на 6). Однако есть центроиды, которые выглядят как гибриды: например, один центроид может одновременно напоминать 8 и 3 или 1 и 7. Это объясняется тем, что KMeans минимизирует внутрикластерное расстояние, а не максимизирует соответствие цифрам, поэтому в один кластер могут попасть образцы разных цифр, визуально похожих.



Скриншот 4 - t-SNE визуализация

Скриншот 4 – t-SNE проекция обучающих данных с раскраской по номерам кластеров KMeans представленная в выводе программы.

t-SNE снижает размерность с 64 до двух компонент, сохраняя локальную структуру. На графике видно несколько сгустков, но они не имеют чётких границ; многие точки одного цвета разбросаны по разным областям, что подтверждает невысокий силуэтный коэффициент. Кластеры перекрываются, и лишь некоторые цвета доминируют в отдельных областях.



Скриншот 5 - *t-SNE* визуализация

Скриншот 5 – та же *t-SNE* проекция в выводе программы, но раскрашенная по истинным цифрам. Здесь отчётливо видны десять областей, хотя между ними есть зоны смешения. Например, цифры 3 и 9 часто находятся рядом, цифры 1 и 7 – тоже. Это соответствует реальной сложности распознавания рукописных цифр и объясняет, почему даже лучшие классификаторы допускают ошибки между этими парами.

Сравнивая скриншоты 4 и 5, можно заметить, что кластеры KMeans лишь грубо аппроксимируют истинные классы: некоторые кластеры захватывают две соседние цифры, а некоторые цифры разбиваются на несколько кластеров.

7. Выводы

В ходе лабораторной работы решены задачи классификации и кластеризации на наборе рукописных цифр. Для классификации использовались перцептрон и стратегия «один против остальных». Показано, что масштабирование признаков с помощью StandardScaler не всегда даёт заметный прирост общей точности (0,9426 против 0,9426 для перцептрона), но оно улучшает сбалансированность ошибок, особенно для классов, которые были проблемными на сырых данных (например, класс 8). Разница между обычным перцептроном и явным OneVsRestClassifier оказалась небольшой, причём встроенная многоклассовая стратегия перцептрона показала даже чуть более высокую точность. Кросс-валидация подтвердила стабильность полученных оценок (около 0,93).

При решении задачи кластеризации KMeans с десятью кластерами получено лишь умеренное соответствие истинным цифрам ($ARI = 0,5083$). Низкий силуэтный коэффициент (0,1216) указывает на то, что данные не образуют естественных компактных групп, соответствующих цифрам. Визуализация центроидов и t-SNE проекций показала, что кластеры KMeans выделяют усреднённые образы, которые иногда являются гибридами нескольких цифр. Тем не менее, кластеризация позволяет выявить общие визуальные паттерны и может использоваться для первоначального анализа структуры данных без учителя.

Все результаты представлены в текстовой, табличной (classification reports, метрики) и графической форме (столбчатая диаграмма точности, матрицы ошибок, центроиды кластеров, t-SNE проекции). Полученные выводы полностью соответствуют теоретическим ожиданиям: линейные классификаторы чувствительны к масштабу признаков, а кластеризация без учителя на сложных визуальных данных даёт лишь приближённое разбиение.

Приложение 1 - Листинг программы

```
"""
Лабораторная работа №4
Вариант: OneVsRestClassifier, Perceptron
Датасет: load_digits (рукописные цифры)
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    silhouette_score,
    adjusted_rand_score,
)
from sklearn.cluster import KMeans
from sklearn.manifold import TSNE

# Конфигурация
RANDOM_STATE = 42
TEST_SIZE = 0.3
MAX_ITER = 1000
TOL = 1e-3
CV_FOLDS = 3
N_CLUSTERS = 10
TSNE_PERPLEXITY = 30

# 1. Загрузка и подготовка данных
def load_and_prepare_data():
    """
    Загружает датасет digits, выводит базовую информацию,
    разделяет на обучающую (70%) и тестовую (30%) выборки
    со стратификацией и масштабирует признаки.
    Возвращает:
        X_train, X_test, y_train, y_test - сырые данные
        X_train_scaled, X_test_scaled - масштабированные данные
        scaler - обученный StandardScaler
    """
    digits = load_digits()
    X, y = digits.data, digits.target

    print("Размерность матрицы признаков:", X.shape)
    print("Уникальные классы:", np.unique(y))
```

```

    print("Распределение классов:\n",
pd.Series(y).value_counts().sort_index())
    print("\nПример признаков первого образца:", X[0])
    print("Тип данных признаков:", X.dtype)

    # Разделение данных
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
    )

    # Масштабирование (StandardScaler)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    return X_train, X_test, y_train, y_test, X_train_scaled, X_test_scaled,
    scaler

# 2. Оценка модели
def evaluate_model(model, X_train, X_test, y_train, y_test,
model_name="Model"):
    """
    Обучает модель и возвращает accuracy, матрицу ошибок и предсказания.
    Также печатает classification report.
    """
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"\n{model_name} - Accuracy: {acc:.4f}")
    print("Classification Report:\n", classification_report(y_test, y_pred))
    cm = confusion_matrix(y_test, y_pred)
    return acc, cm, y_pred

#3. Обучение и сравнение моделей
def train_and_evaluate_models(X_train, X_test, y_train, y_test,
dataset_name="сырые данные"):
    """
    Обучает Perceptron и OneVsRestClassifier(Perceptron) на переданных
    данных.
    Возвращает словарь с результатами.
    """
    # Perceptron
    perceptron = Perceptron(max_iter=MAX_ITER, random_state=RANDOM_STATE,
tol=TOL)
    acc_perc, cm_perc, _ = evaluate_model(
        perceptron, X_train, X_test, y_train, y_test,
        f"Perceptron ({dataset_name})"
    )

    # OneVsRestClassifier + Perceptron

```



```

    ovr = OneVsRestClassifier(Perceptron(max_iter=MAX_ITER,
random_state=RANDOM_STATE, tol=TOL))
    acc_ovr, cm_ovr, _ = evaluate_model(
        ovr, X_train, X_test, y_train, y_test,
        f"OneVsRestClassifier + Perceptron ({dataset_name})"
    )

    return {"perceptron": (acc_perc, cm_perc), "ovr": (acc_ovr, cm_ovr)}

# 4. Кросс-валидация
def cross_validate_models(X_train, y_train):
    """
    Выполняет 3-кратную кросс-валидацию для двух моделей
    на масштабированных данных и печатает результаты.
    """
    kf = KFold(n_splits=CV_FOLDS, shuffle=True, random_state=RANDOM_STATE)

    perceptron = Perceptron(max_iter=MAX_ITER, random_state=RANDOM_STATE,
tol=TOL)
    cv_scores_perc = cross_val_score(perceptron, X_train, y_train, cv=kf,
scoring='accuracy')

    ovr = OneVsRestClassifier(Perceptron(max_iter=MAX_ITER,
random_state=RANDOM_STATE, tol=TOL))
    cv_scores_ovr = cross_val_score(ovr, X_train, y_train, cv=kf,
scoring='accuracy')

    print(f"\nPerceptron CV accuracy: {cv_scores_perc.mean():.4f} (+/-
{cv_scores_perc.std():.4f})")
    print(f"OneVsRestClassifier CV accuracy: {cv_scores_ovr.mean():.4f} (+/-
{cv_scores_ovr.std():.4f})")

# 5. Визуализация
def plot_accuracy_comparison(accuracies, model_names):
    """
    Строит столбчатую диаграмму сравнения точности моделей.
    accuracies: список float, model_names: список str.
    """
    plt.figure(figsize=(8, 5))
    bars = plt.bar(model_names, accuracies, color=['gray', 'lightblue',
'gray', 'lightblue'])
    bars[2].set_color('steelblue')
    bars[3].set_color('steelblue')
    plt.ylim(0, 1)
    plt.ylabel('Accuracy')
    plt.title('Сравнение точности классификаторов на сырых и масштабированных
данных')
    for bar, acc in zip(bars, accuracies):
        plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{acc:.3f}', ha='center', va='bottom')
    plt.grid(axis='y', linestyle='--', alpha=0.7)

```

```

plt.tight_layout()
plt.show()

def plot_confusion_matrices(cm_raw, cm_scaled, title_raw="OvR, сырые данные",
title_scaled="OvR, масштабированные данные"):
    """
    Отображает две тепловые карты матриц ошибок.
    """
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    sns.heatmap(cm_raw, annot=True, fmt='d', cmap='Blues', ax=axes[0])
    axes[0].set_title(f'Матрица ошибок ({title_raw})')
    axes[0].set_xlabel('Предсказанный класс')
    axes[0].set_ylabel('Истинный класс')

    sns.heatmap(cm_scaled, annot=True, fmt='d', cmap='Greens', ax=axes[1])
    axes[1].set_title(f'Матрица ошибок ({title_scaled})')
    axes[1].set_xlabel('Предсказанный класс')
    axes[1].set_ylabel('Истинный класс')
    plt.tight_layout()
    plt.show()

# 6. Кластеризация и визуализация
def perform_clustering(X_train_scaled, y_train):
    """
    Выполняет кластеризацию KMeans, оценивает качество (silhouette, ARI),
    визуализирует центроиды и строит t-SNE проекции.
    """
    # KMeans
    kmeans = KMeans(n_clusters=N_CLUSTERS, random_state=RANDOM_STATE,
n_init='auto')
    clusters = kmeans.fit_predict(X_train_scaled)

    # Оценка
    sil_score = silhouette_score(X_train_scaled, clusters)
    ari_score = adjusted_rand_score(y_train, clusters)
    print(f"\nSilhouette Score: {sil_score:.4f}")
    print(f"Adjusted Rand Index (ARI): {ari_score:.4f}")

    # Центроиды кластеров
    fig, axes = plt.subplots(2, 5, figsize=(10, 4))
    centroids = kmeans.cluster_centers_
    for i, ax in enumerate(axes.flat):
        ax.imshow(centroids[i].reshape(8, 8), cmap='gray')
        ax.set_title(f'Кластер {i}')
        ax.axis('off')
    plt.suptitle('Центроиды кластеров KMeans')
    plt.tight_layout()
    plt.show()

    # t-SNE проекция
    print("Вычисление t-SNE проекции (может занять несколько секунд)...")

```

```

    tsne = TSNE(n_components=2, random_state=RANDOM_STATE,
perplexity=TSNE_PERPLEXITY)
    X_tsne = tsne.fit_transform(X_train_scaled)

    # Цвет по кластерам
    plt.figure(figsize=(10, 6))
    scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=clusters,
сmap='tab10', alpha=0.7)
    plt.colorbar(scatter, label='Номер кластера')
    plt.title('t-SNE визуализация результатов кластеризации KMeans (10
кластеров)')
    plt.xlabel('t-SNE компонента 1')
    plt.ylabel('t-SNE компонента 2')
    plt.show()

    # Цвет по истинным меткам
    plt.figure(figsize=(10, 6))
    scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_train,
сmap='tab10', alpha=0.7)
    plt.colorbar(scatter, label='Истинная цифра')
    plt.title('t-SNE визуализация: истинные метки классов')
    plt.xlabel('t-SNE компонента 1')
    plt.ylabel('t-SNE компонента 2')
    plt.show()

#Основной блок
def main():
    # Загрузка и подготовка данных
    (X_train, X_test, y_train, y_test,
    X_train_scaled, X_test_scaled, _) = load_and_prepare_data()

    # Обучение на сырых данных
    print("ОБУЧЕНИЕ НА СЫРЫХ ДАННЫХ (без масштабирования)")
    raw_results = train_and_evaluate_models(X_train, X_test, y_train, y_test,
"сырые данные")

    # Обучение на масштабированных данных
    print("ОБУЧЕНИЕ НА МАСШТАБИРОВАННЫХ ДАННЫХ (StandardScaler)")
    scaled_results = train_and_evaluate_models(X_train_scaled, X_test_scaled,
y_train, y_test, "масштабированные данные")

    # Сбор метрик для визуализации
    accuracies = [
        raw_results["perceptron"][0],    # Perceptron raw
        raw_results["ovr"][0],           # OvR raw
        scaled_results["perceptron"][0], # Perceptron scaled
        scaled_results["ovr"][0]         # OvR scaled
    ]
    model_names = ['Perceptron raw', 'OvR raw', 'Perceptron scaled', 'OvR
scaled']
    plot_accuracy_comparison(accuracies, model_names)

```

```
# Матрицы ошибок (для OvR моделей)
plot_confusion_matrices(
    raw_results["ovr"][1],
    scaled_results["ovr"][1],
    "OvR, сырые данные",
    "OvR, масштабированные данные"
)

# Кросс-валидация на масштабированных данных
print("КРОСС-ВАЛИДАЦИЯ (3-fold) на масштабированных данных")
cross_validate_models(X_train_scaled, y_train)

# Кластеризация и визуализация
perform_clustering(X_train_scaled, y_train)

if __name__ == "__main__":
    main()
```